

SIMATS ENGINEERING

ASSESSMENT TOOL 3: Reverse Engineering Task

COURSE NAME:	Cloud Computing and Big Data Analytics	COURSE CODE:	CSA1522
---------------------	---	---------------------	---------

COURSE OUTCOME COVERED

CO4: *Analyze big data characteristics and challenges across domains including security and application aspects.*
(BL4)

Dave's Taxonomy: Articulation (Level 4)
Question Count: 5

Total Marks: 30

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____

Assessment Tasks

Reverse Engineering Task

For each task, study the described big data solution, infer how it was built, identify the underlying components, and explain what design decisions were likely made.

1. A big data platform collects billions of website clicks, mobile interactions, and customer transactions every day. Reverse engineer the probable distributed storage system, ingestion framework, and batch-processing tools used. Infer how structured and semi-structured data are separated and indexed for analysis. Identify the likely stream-processing engines supporting real-time analytics and reporting. Explain the scalability and partitioning strategies probably implemented for handling massive workloads. Describe how dashboards and business intelligence systems are likely connected to the analytics layer.
2. An e-commerce platform tracks cart additions, abandoned checkouts, product views, and payment behavior across regions. Reverse engineer the likely event streaming tools, customer profiling databases, and recommendation workflow used. Infer how session data, clickstream logs, and transaction records are synchronized for real-time personalization. Identify probable caching systems, distributed query engines, and machine learning stages involved. Explain how structured order data and semi-structured browsing data are merged for analytics. Deduce the scalability and fault-tolerance decisions likely implemented. Describe how dashboards and KPI monitoring are probably generated from the pipeline.
3. A big data healthcare system processes patient histories, diagnostic images, wearable sensor streams, and prescription records. Reverse engineer the probable hybrid database architecture managing structured and unstructured healthcare information. Infer the likely data integration tools connecting hospitals, laboratories, and insurance systems. Identify the streaming and machine learning frameworks probably used for disease prediction and monitoring. Explain how privacy, encryption, and compliance requirements are likely enforced in the platform. Describe the analytics pipeline that may support clinical decision-making and population health studies.
4. A smart city platform gathers traffic sensor feeds, CCTV metadata, weather updates, and public transport activity continuously. Reverse engineer the probable edge computing setup, distributed

messaging system, and centralized storage design. Infer how geospatial analytics and streaming computations are handled for congestion prediction. Identify the likely databases used for time-series and location-aware queries. Explain how batch and real-time analytics are combined for urban planning insights. Deduce the security and access-control measures likely protecting citizen and infrastructure data. Describe how visualization systems probably deliver live operational intelligence.

5. A healthcare analytics system integrates patient records, wearable device readings, lab reports, and appointment histories. Reverse engineer the probable hybrid storage architecture handling sensitive structured and unstructured medical data. Infer the likely ETL pipelines and interoperability standards connecting hospitals and clinics. Identify how real-time monitoring and predictive health analytics are probably implemented. Explain the role of distributed processing frameworks in population-level analysis. Deduce the encryption, compliance, and audit mechanisms likely enforced for data governance. Describe how machine learning models may support diagnosis risk scoring and treatment recommendations.

Reverse Engineering Task Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Inference Accuracy	25%	Infers system components and flow with high accuracy	Mostly accurate inference	Basic inference	Weak inference
Understanding of Architecture	25%	Shows clear understanding of underlying big data architecture	Good understanding	Partial understanding	Poor understanding
Use of Technical Evidence	20%	Uses strong reasoning and technical evidence	Reasonable evidence	Limited evidence	No meaningful evidence
Depth of Analysis	15%	Analysis is deep and insightful	Reasonably deep	Basic depth	Superficial analysis
Clarity	15%	Clear and organized explanation	Generally clear	Somewhat unclear	Poorly presented

1. A big data platform collects billions of website clicks, mobile interactions, and customer transactions every day. Reverse engineer the probable distributed storage system, ingestion framework, and batch-processing tools used. Infer how structured and semi-structured data are separated and indexed for analysis. Identify the likely stream-processing engines supporting real-time analytics and reporting. Explain the scalability and partitioning strategies probably implemented for handling massive workloads. Describe how dashboards and business intelligence systems are likely connected to the analytics layer.

Reverse Engineering a Website-Scale Big Data Platform:

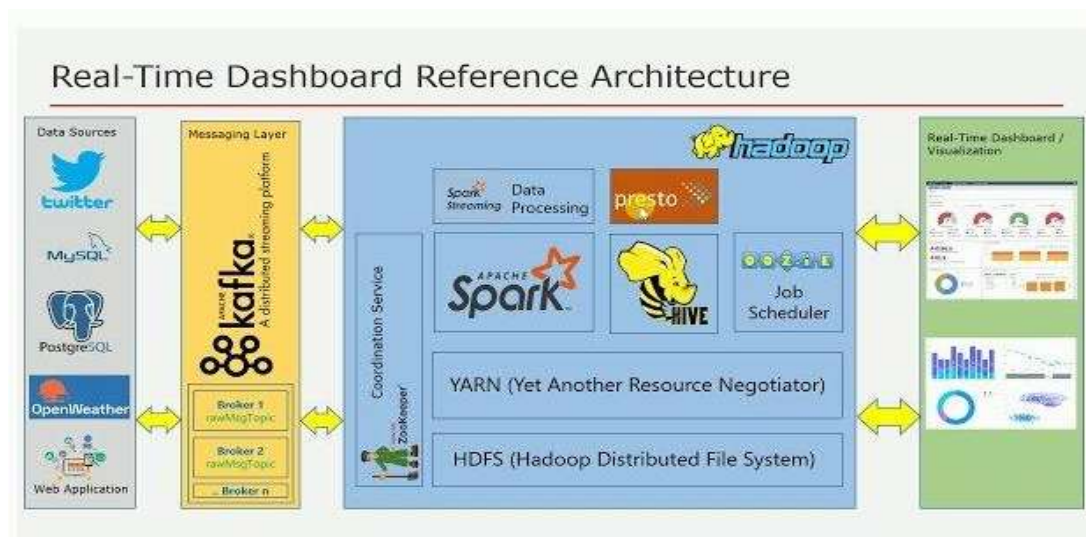
Problem Statement:

A platform that collects billions of website clicks, mobile interactions, and customer transactions every day needs an architecture that can ingest massive volumes of data continuously, process it with low latency, and store it efficiently for both historical reporting and real-time analytics. Although the exact implementation is unknown, the requirements strongly suggest a modern distributed data platform built around event streaming, distributed storage, and parallel processing.

The storage layer is divided into :

- **Data Lake:** Stores raw clickstreams, mobile events, logs, and transaction records. Eg : HDFS , Amazon S3, Azure Lake.
- **Distributed NoSQL databases :** Handle fast reads and writes for operational analytics. Eg; Apache Cassandra or HBase
- **Data Warehouse:** Stores curated datasets for reporting. Eg : Snowflake, BigQuery, or Redshift

Probable Ingestion Framework:



Different producers continuously send:

- Website clicks
- Mobile app events
- Payment transactions
- User interactions

For scheduled imports such as CRM records or historical sales, Apache NiFi or Apache Airflow would likely manage batch ingestion.

Batch Processing Tools

The strongest candidate is Apache Spark.

Spark can:

- Aggregate billions of events
- Build customer profiles
- Generate daily reports
- Train machine learning models

Stream Processing Engine

Real-time reporting requires continuous processing.

Likely technologies include:

- Apache Spark Streaming
- Apache Flink

Flink is particularly suitable when very low latency is required because it processes events continuously rather than waiting for micro-batches.

Scalability and Partitioning Strategy

The platform probably scales horizontally.

Key strategies include:

- Kafka topic partitioning
- Spark distributed partitions
- Data partitioning by date, region, or customer ID
- Automatic cluster scaling using Kubernetes

Instead of upgrading one server, additional nodes can simply join the cluster.

Dashboard and BI Integration

Business users rarely access raw data directly.

Instead:

1. Spark writes aggregated datasets.
2. The warehouse stores optimized tables.
3. Tableau or Power BI queries those tables.
4. Executives view KPIs through live dashboards.

Examples include:

- Daily revenue
- Active users
- Conversion rate
- Regional sales

This architecture supports both historical reporting and real-time operational decisions while remaining highly scalable.

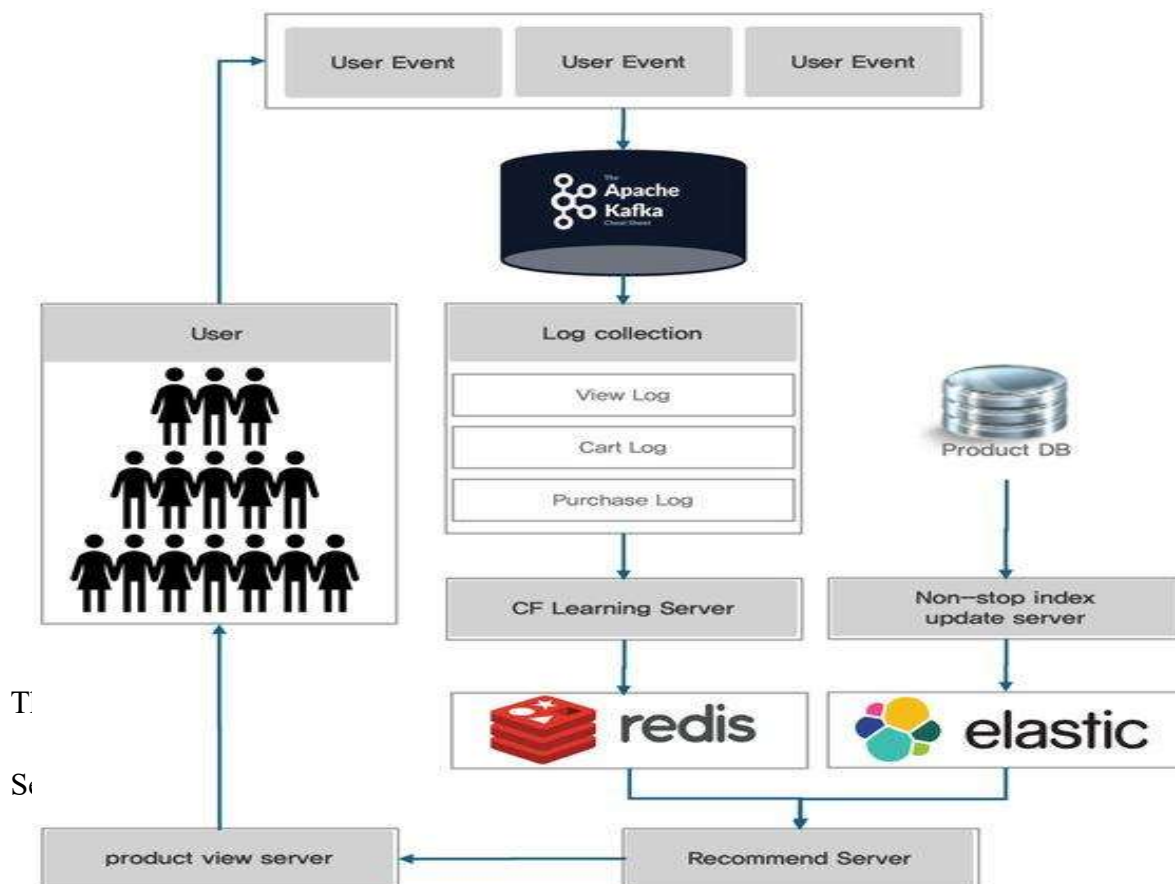
2. An e-commerce platform tracks cart additions, abandoned checkouts, product views, and payment behavior across regions. Reverse engineer the likely event streaming tools, customer profiling databases, and recommendation workflow used. Infer how session data, clickstream logs, and transaction records are synchronized for real-time personalization. Identify probable caching systems, distributed query engines, and machine learning stages involved. Explain how structured order data and semi-structured browsing data are merged for analytics. Deduce the scalability and fault-tolerance decisions likely implemented. Describe how dashboards and KPI monitoring are probably generated from the pipeline.

Reverse Engineering an E-Commerce Personalization Platform

Problem Statement:

An e-commerce company records cart additions, abandoned checkouts, product views, and payments across multiple regions. The platform must synchronize customer activity instantly so recommendations remain relevant while users continue browsing.

Probable Event Streaming Architecture:



- Cart updates
- Checkout events
- Payments
- Search activity

Kafka's partitioned design allows millions of simultaneous events.

Synchronizing Session, Clickstream, and Transactions

The platform probably creates a unique customer identifier.

Every interaction receives:

- Session ID
- Customer ID
- Timestamp

Streaming processors merge these events into one customer timeline.

For example:

A customer views shoes, adds them to the cart, abandons checkout, and later returns. The recommendation engine immediately recognizes the previous activity.

Feature Engineering

Customer behavior features include:

- Purchase frequency
- Product categories
- Time spent browsing

Ranking Model

Likely algorithms include:

- Collaborative filtering

- Gradient boosting
- Deep learning ranking models

The ranking stage produces personalized product lists.

Caching System

Real-time recommendations cannot repeatedly query large databases.

The platform would likely use Redis.

Redis stores:

- Active sessions
- Popular products
- Recent recommendations

This dramatically reduces response time.

Distributed Query Engine

Analysts need fast access to large datasets.

Likely tools include:

- Trino
- Presto

These engines query data across multiple storage systems without moving everything into one database.

Fault Tolerance

The architecture probably includes:

- Kafka replication
- Spark checkpointing
- Multi-region backups
- Automatic node replacement

If one server fails, another continues processing.

Dashboard Generation

Business dashboards likely display:

- Cart abandonment rate
- Conversion rate
- Revenue by region
- Recommendation performance

Power BI or Tableau would query warehouse tables refreshed continuously through the streaming pipeline.

3.A big data healthcare system processes patient histories, diagnostic images, wearable sensor streams, and prescription records. Reverse engineer the probable hybrid database architecture managing structured and unstructured healthcare information. Infer the likely data integration tools connecting hospitals, laboratories, and insurance systems. Identify the streaming and machine learning frameworks probably used for disease prediction and monitoring. Explain how privacy, encryption, and compliance requirements are likely enforced in the platform. Describe the analytics pipeline that may support clinical decision-making and population health studies.

Reverse Engineering a Healthcare Big Data Platform

Problem Analysis

A healthcare platform combines patient histories, medical images, wearable sensors, prescriptions, laboratories, and insurance systems. This requires both real-time patient monitoring and secure long-term medical record management.

Data Integration

Hospitals rarely use identical software.

The platform therefore likely relies on interoperability standards.

Most likely:

- HL7

- FHIR APIs
- DICOM

These standards allow hospitals, laboratories, and insurers to exchange information consistently.

Streaming Framework

Wearable devices continuously generate:

- Heart rate
- Blood pressure
- Oxygen levels

Apache Kafka probably receives these events.

Apache Spark Streaming or Flink analyzes them immediately.

Machine Learning Framework

Healthcare prediction likely involves:

- TensorFlow
- PyTorch
- Spark MLlib

Possible models include:

- Disease risk prediction
- Readmission prediction
- Early warning detection

Example:

An abnormal heart-rate pattern triggers an alert before symptoms worsen.

Privacy and Compliance

Healthcare platforms require extremely strong security.

Likely protections include:

- AES-256 encryption
- TLS transmission
- Role-based access control
- Multi-factor authentication
- Audit logging

Compliance would likely align with regulations such as HIPAA or national healthcare privacy laws.

Analytics Pipeline

The pipeline probably follows this sequence:

1. Collect hospital data.
2. Integrate through FHIR.
3. Stream wearable data.
4. Store raw data.
5. Process using Spark.
6. Generate predictions.
7. Display clinician dashboards.

Population health studies would analyze aggregated, de-identified records rather than exposing individual patient information.

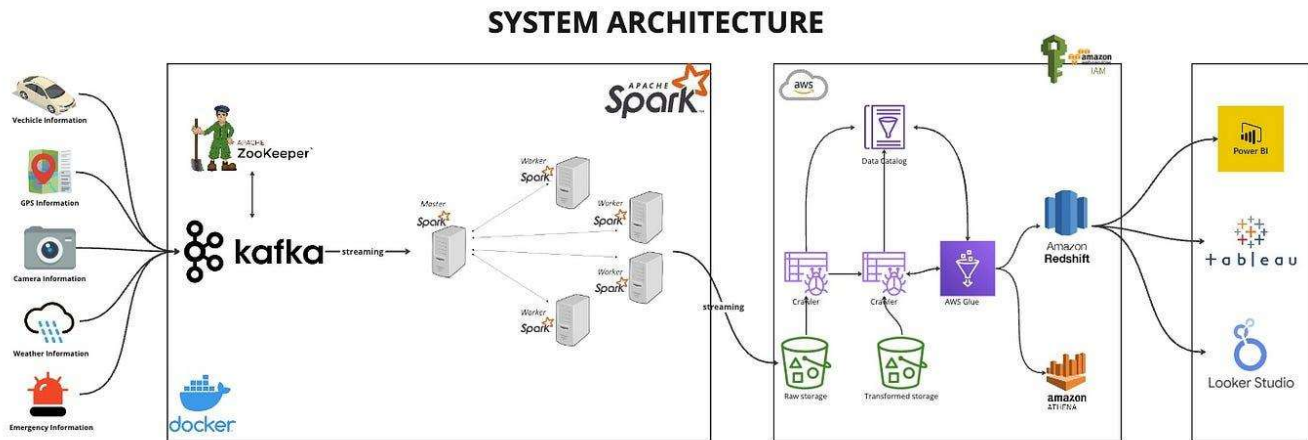
4.A smart city platform gathers traffic sensor feeds, CCTV metadata, weather updates, and public transport activity continuously. Reverse engineer the probable edge computing setup, distributed messaging system, and centralized storage design. Infer how geospatial analytics and streaming computations are handled for congestion prediction. Identify the likely databases used for time-series and location-aware queries. Explain how batch and real-time analytics are combined for urban planning insights. Deduce the security and access-control measures likely protecting citizen and infrastructure data. Describe how visualization systems probably deliver live operational intelligence.

Reverse Engineering a Smart City Platform

Problem Analysis

A smart city continuously receives traffic sensor data, CCTV metadata, weather updates, and public transport activity. Because thousands of devices generate information simultaneously, much of the processing likely begins near the data source before reaching centralized systems.

Probable Edge Computing Setup



- Video preprocessing
- Sensor filtering
- Local event detection

Instead of transmitting entire video streams, cameras may send metadata describing detected vehicles.

This reduces bandwidth significantly.

Distributed Messaging System

Apache Kafka is the strongest candidate.

Different topics likely exist for:

- Traffic sensors
- Weather stations
- Public transport

- CCTV metadata

Kafka allows independent processing of each data stream.

Geospatial Analytics

Traffic prediction requires location-aware processing.

Likely technologies include:

- Apache Sedona
 - PostGIS
- These systems perform operations such as:
- Route calculation
 - Congestion mapping
 - Geographic clustering

Streaming Computation

Apache Flink is particularly suitable because congestion prediction requires continuous event processing.

Example:

A sudden increase in vehicle counts automatically updates traffic predictions.

Time-Series Database

Traffic measurements naturally fit time-series storage.

Likely databases include:

- InfluxDB
- TimescaleDB

These efficiently store timestamped sensor readings.

Combining Batch and Real-Time Analytics

Two analytical paths likely operate simultaneously.

Real-time

- Live congestion
- Incident detection
- Signal adjustments

Batch

- Monthly traffic studies
- Infrastructure planning
- Seasonal analysis

Together they support immediate operations and long-term urban planning.

Security Measures

Because city infrastructure is sensitive, protections likely include:

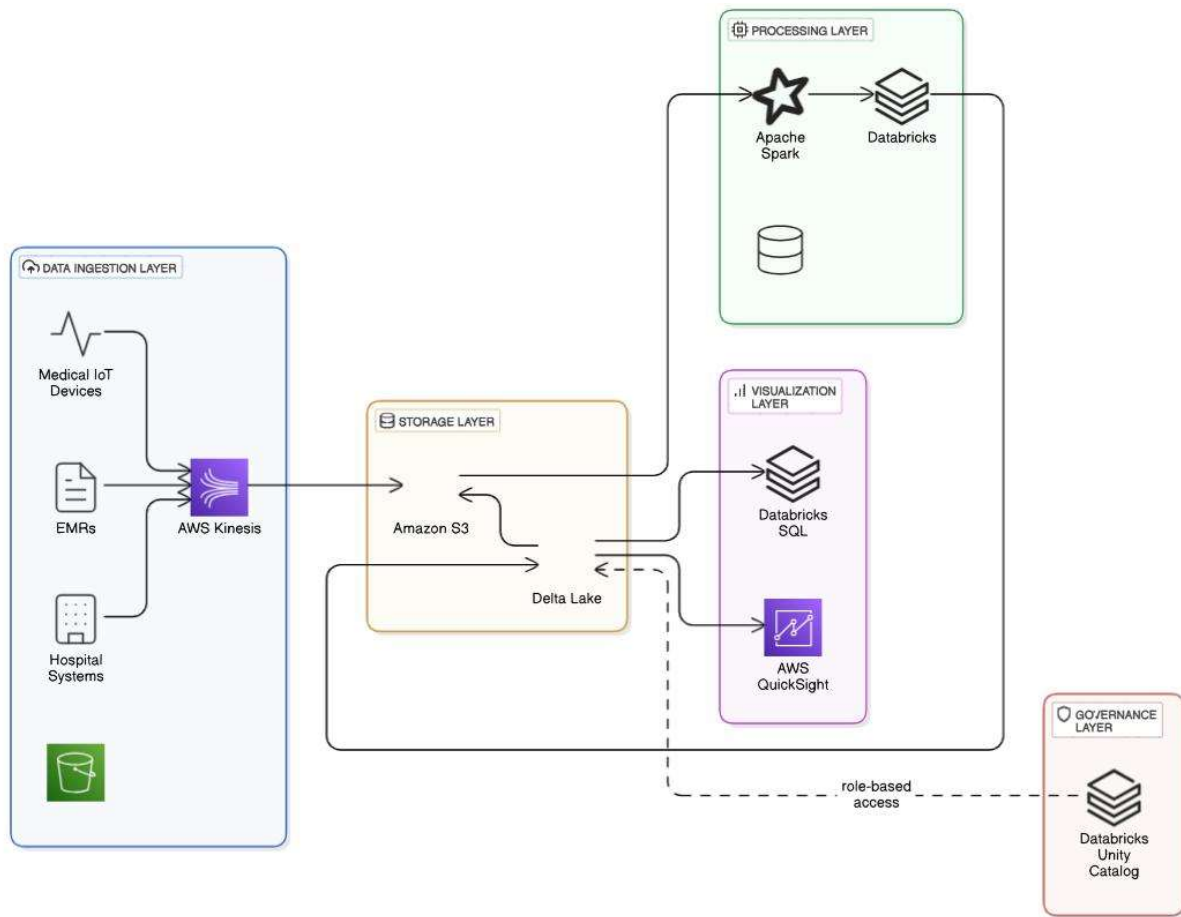
- Device authentication
- Encrypted communication
- Network segmentation
- Role-based permissions
- Continuous monitoring

Visualization

Operations centers likely use GIS dashboards showing:

- Live traffic
- Bus movement
- Weather alerts
- Road incidents

These dashboards update continuously from streaming analytics.



Probable Hybrid Storage Architecture

ETL Pipeline

The platform likely performs:

1. Data extraction from hospitals.
2. Standardization through FHIR.
3. Validation.
4. Transformation.
5. Loading into the Data Lake.

Apache NiFi or Airflow would likely automate these workflows.

Interoperability Standards

Healthcare organizations probably exchange information using:

- FHIR

- HL7
- DICOM

These standards reduce compatibility problems across different hospitals.

Real-Time Monitoring

Wearable devices continuously transmit health measurements.

Kafka collects these events.

Spark Streaming analyzes:

- Heart rate
- Oxygen levels
- Activity patterns

Abnormal readings immediately generate alerts.

Distributed Processing

Population-level analysis requires enormous computational power.

Apache Spark distributes work across multiple servers, making it possible to analyze millions of patient records simultaneously.

Example applications include:

- Disease prevalence
- Treatment effectiveness
- Regional health trends

Encryption and Governance

Strong governance likely includes:

- Encryption at rest
- TLS encryption

- Role-based access
- Audit trails
- Consent management

Every access to patient records is likely logged for accountability.

Machine Learning Applications

The platform probably supports:

- Diagnosis risk scoring
- Hospital readmission prediction
- Treatment recommendation
- Patient prioritization

Rather than replacing doctors, these models provide decision-support tools that help clinicians identify high-risk patients earlier.